

# Queue

---

## Queue

- 1. Queue Overview
  - Why Do We Need Queues?
  - Typical Application Scenarios
- 2. Core Concepts
  - 2.1 How Queues Work
  - 2.2 Queue Creation API
  - 2.3 Send API
  - 2.4 Receive API
  - 2.5 Blocking vs Non-Blocking Comparison
- 3. Project Architecture and Flow
- 4. Hardware Dependencies
- 5. Source Code Analysis
  - 5.1 Header Files and Macro Configuration
  - 5.2 prv\_button\_level\_stable() — Debounce Detection
  - 5.3 vSendTask() — Send Task
  - 5.4 vRecvTask() — Receive Task
  - 5.5 prv\_running\_horse() — Marquee Effect
  - 5.6 app\_main() — Main Entry
- 6. Operation Flow
  - 6.1 Compilation and Flashing Flow
  - 6.2 Normal Operation
- 7. FAQ

---

## 1. Queue Overview

**Queue** is FreeRTOS's most fundamental inter-task communication mechanism. It is a **FIFO (First In First Out)** data buffer where one task writes data to the queue and another task reads data from it, achieving data transfer and flow control between tasks.

### Why Do We Need Queues?

In a multi-tasking system, tasks often need to exchange data. If two tasks directly operate on the same global variable, they face **race conditions** (one task gets interrupted halfway through writing, causing data corruption). Queues use **built-in internal mutex protection** to ensure atomicity of data writes and reads, fundamentally avoiding these problems.

### Typical Application Scenarios

| Scenario              | Sender                      | Receiver           | Queue Function                             |
|-----------------------|-----------------------------|--------------------|--------------------------------------------|
| Button event delivery | GPIO interrupt/polling task | UI processing task | Buffer button events, no loss, no blocking |

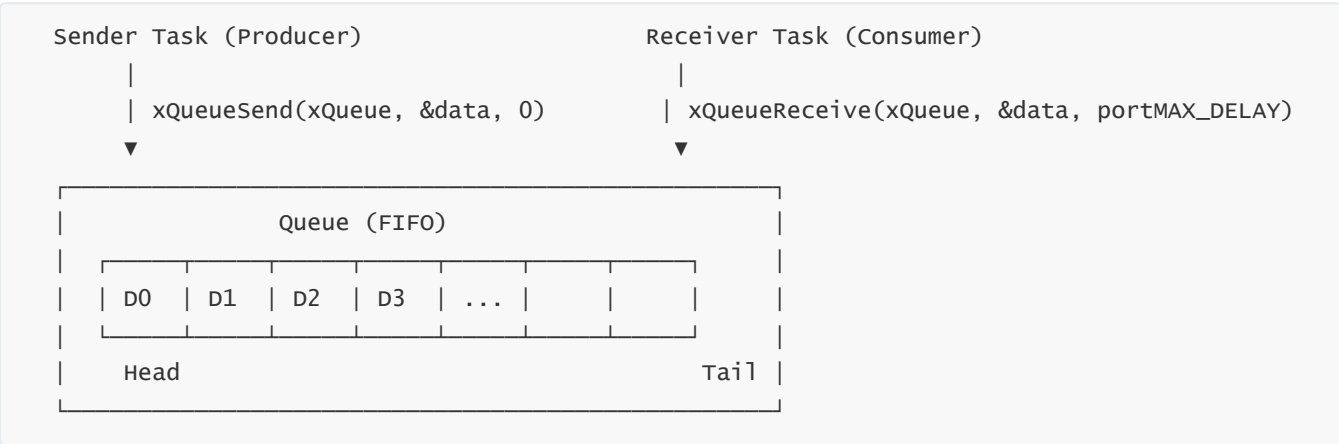
| Scenario                | Sender                 | Receiver                     | Queue Function                                                          |
|-------------------------|------------------------|------------------------------|-------------------------------------------------------------------------|
| Sensor data acquisition | ADC sampling task      | Data processing/display task | Decouple sampling rate from computation rate                            |
| UART command parsing    | UART receive task      | Command execution task       | Buffer command frames, process frame by frame                           |
| Network data forwarding | WiFi receive task      | Business logic task          | Isolate network stack from business logic                               |
| Log output              | Various business tasks | Log writing task             | Centralized log management, avoid multi-task contention for peripherals |

**Core Value:** Queues decouple "producers" and "consumers" — both can run at different frequencies, on different CPU cores, without interfering with each other. Producers just stuff data into the queue, consumers take data at their own pace, backpressure naturally occurs when the queue is full, and consumers block and wait when the queue is empty.

This project demonstrates queue **creation, sending, and receiving**: each button press sends one data item to the queue (green LED lights for 300ms), the receiving task takes one data item every 1 second, and after receiving 30 items, it enters an infinite rainbow marquee loop.

## 2. Core Concepts

### 2.1 How Queues Work



### 2.2 Queue Creation API

```
QueueHandle_t xQueueCreate(  
    UBaseType_t uxQueueLength, // Queue length (max items)  
    UBaseType_t uxItemSize     // Item size (bytes)  
);
```

Returns NULL on failure (insufficient memory).

## 2.3 Send API

```
BaseType_t xQueueSend(  
    QueueHandle_t xQueue,           // Queue handle  
    const void *pvItemToQueue,     // Data pointer  
    TickType_t xTicksToWait        // Timeout (ticks)  
);
```

| xTicksToWait                    | Behavior                                                                       |
|---------------------------------|--------------------------------------------------------------------------------|
| 0                               | Non-blocking — returns <code>errQUEUE_FULL</code> immediately if queue is full |
| <code>portMAX_DELAY</code>      | Permanent block — waits until queue has space                                  |
| <code>pdMS_TO_TICKS(100)</code> | 100ms timeout — returns error after timeout                                    |

Must use `xQueueSendFromISR()` in ISR.

## 2.4 Receive API

```
BaseType_t xQueueReceive(  
    QueueHandle_t xQueue,           // Queue handle  
    void *pvBuffer,                // Receive buffer  
    TickType_t xTicksToWait        // Timeout (ticks)  
);
```

## 2.5 Blocking vs Non-Blocking Comparison

| Mode                 | xTicksToWait               | Queue Full/Empty Behavior  | Applicable Scenario   |
|----------------------|----------------------------|----------------------------|-----------------------|
| Non-blocking send    | 0                          | Return error immediately   | Cannot wait scenarios |
| Blocking send        | <code>portMAX_DELAY</code> | Wait until space available | Data must not be lost |
| Non-blocking receive | 0                          | Return error immediately   | Polling               |
| Blocking receive     | <code>portMAX_DELAY</code> | Wait until data available  | Event-driven          |

### 3. Project Architecture and Flow

```
app_main()
├─ gpio_config()           ← Button GPIO0 (pull-up input)
├─ led_strip_init()        ← WS2812 (GPIO48, RMT 10MHz)
├─ xQueueCreate(30, sizeof(uint32_t)) ← Queue capacity 30
├─ xTaskCreate(vSendTask) → Core 1, Prio 2 (Send task)
└─ xTaskCreate(vRecvTask) → Core 0, Prio 2 (Receive task)
```

GPIO0 pressed → vSendTask —(uint32\_t)—→ xQueue → vRecvTask

↓  
Green LED 300ms (send success)

↓  
Take 1 data every 1s  
After 30 received → infinite marquee

Queue Parameters:

| Parameter    | Value                    | Meaning                                |
|--------------|--------------------------|----------------------------------------|
| Queue length | 30                       | Max 30 uint32_t data items buffered    |
| Item size    | 4 bytes                  | sizeof(uint32_t)                       |
| Send mode    | Non-blocking (timeout=0) | Drop and print warning when queue full |
| Receive mode | Blocking (portMAX_DELAY) | Task stays Blocked until data arrives  |

### 4. Hardware Dependencies

| Pin     | Function       | Description                  |
|---------|----------------|------------------------------|
| GPIO 0  | BOOT button    | Internal pull-up, active low |
| GPIO 48 | WS2812 RGB LED | RMT peripheral driver        |

**Required Components:** `freertos/FreeRTOS`, `freertos/task`, `freertos/queue`, `driver/gpio`, `led_strip`, `esp_log`

This project uses a **third-party library** again: For details, review (**Third-party library installation process: 2.Basic Peripherals → 10.RGB LED Effects**)

**Note:** `led_strip` is a third-party component provided by ESP-IDF Component Registry, not built into IDF. Before use, declare the dependency in `main/idf_component.yml`, otherwise compilation will fail to find the header file:

```
## IDF Component Manager Manifest File
dependencies:
idf:
  version: '>=4.1.0'
espressif/led_strip: ^3.0.3
```

After declaration, execute `idf.py build`, and the component manager will automatically download the component to the `managed_components/` directory.

## 5. Source Code Analysis

Full source code: [Source Code](#) → [ESP-IDF](#) → [4.FreRTOS](#) → [2. RTOS\\_Inter-task\\_communication](#) → [RTOS-QUE](#)

### 5.1 Header Files and Macro Configuration

This section defines all adjustable parameters such as task priority, stack size, queue capacity, button debounce parameters, and WS2812 LED pins, and declares global handle variables. All macros are concentrated at the file header for easy unified adjustment without modifying deep code.

```
#include <stdbool.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "led_strip.h"

// Log tag
static const char *TAG = "queue_demo";

/* ===== Task Config ===== */

#define SEND_TASK_PRIORITY      2
#define RECV_TASK_PRIORITY     2
#define SEND_TASK_STACK_SIZE   2048
#define RECV_TASK_STACK_SIZE   2048

/* ===== Queue Config ===== */

#define QUEUE_LENGTH            30
#define MAX_SEND_COUNT         30
#define MAX_RECV_COUNT         30

/* ===== Button Config ===== */

#define BUTTON_GPIO             0
#define DEBOUNCE_MS            30
#define DEBOUNCE_SAMPLES       3

/* ===== LED Config ===== */

#define LED_GPIO                48
#define LED_NUM                 1
#define LED_BRIGHTNESS         80

/* ===== Global Handles ===== */

static QueueHandle_t xQueue = NULL;
```

```
static led_strip_handle_t led_strip;
static uint32_t send_count = 0;
static uint32_t recv_count = 0;
```

## 5.2 prv\_button\_level\_stable() — Debounce Detection

Continuously samples GPIO level multiple times, only returns true if all samples are consistent. More reliable than single-delay debouncing (2.KEY example).

```
static bool prv_button_level_stable(uint32_t u1Level)
{
    for (int i = 0; i < DEBOUNCE_SAMPLES; i++)
    {
        if (gpio_get_level(BUTTON_GPIO) != u1Level)
        {
            return false;
        }
        vTaskDelay(pdMS_TO_TICKS(DEBOUNCE_MS / DEBOUNCE_SAMPLES));
    }
    return true;
}
```

## 5.3 vSendTask() — Send Task

Polls BOOT button, sends incremental data to queue after debouncing (non-blocking), WS2812 green LED lights for 300ms on successful send.

```
static void vSendTask(void *pvParameters)
{
    uint32_t send_data = 0;
    (void)pvParameters;

    while (send_count < MAX_SEND_COUNT)
    {
        while (gpio_get_level(BUTTON_GPIO) == 1)
        {
            vTaskDelay(pdMS_TO_TICKS(10));
        }

        if (!prv_button_level_stable(0))
        {
            continue;
        }

        if (xQueueSend(xQueue, &send_data, 0) == pdPASS)
        {
            send_count++;
            send_data++;
            ESP_LOGI(TAG, "Send OK: %lu | sent: %lu/%d",
                     send_data - 1, send_count, MAX_SEND_COUNT);
        }
    }
}
```

```

        led_strip_set_pixel(led_strip, 0, 0, LED_BRIGHTNESS, 0);
        led_strip_refresh(led_strip);
        vTaskDelay(pdMS_TO_TICKS(300));
        led_strip_clear(led_strip);
        led_strip_refresh(led_strip);
    }
    else
    {
        ESP_LOGW(TAG, "Queue FULL, send failed");
    }

    while (gpio_get_level(BUTTON_GPIO) == 0)
    {
        vTaskDelay(pdMS_TO_TICKS(10));
    }
    prv_button_level_stable(1);
}

ESP_LOGI(TAG, "send task STOP. Total sent: %lu", send_count);
vTaskDelete(NULL);
}

```

**Why use non-blocking for send (timeout=0)?** Button events should not block waiting for space. When queue is full, simply drop and print warning, don't block button handling.

## 5.4 vRecvTask() — Receive Task

Blocks waiting for queue data, delays 1s after taking each item. After receiving 30, enters infinite rainbow marquee loop.

```

static void vRecvTask(void *pvParameters)
{
    uint32_t recv_data;
    (void)pvParameters;

    while (recv_count < MAX_RECV_COUNT)
    {
        if (xQueueReceive(xQueue, &recv_data, portMAX_DELAY) == pdPASS)
        {
            recv_count++;
            ESP_LOGI(TAG, "Receive: %lu | recv: %lu/%d",
                     recv_data, recv_count, MAX_RECV_COUNT);
        }

        vTaskDelay(pdMS_TO_TICKS(1000));
    }

    ESP_LOGI(TAG, "Receive task STOP. Total received: %lu", recv_count);

    ESP_LOGI(TAG, "Running horse light show start...");
    while (1)
    {

```

```

        prv_running_horse(5);
    }
}

```

**Why use blocking for receive (portMAX\_DELAY)?** When queue is empty, receive task enters Blocked state, not consuming CPU. Once data arrives in queue, scheduler immediately wakes it up. This is the most energy-efficient waiting method in FreeRTOS.

## 5.5 prv\_running\_horse() — Marquee Effect

Cycles through 7 rainbow colors on WS2812, parameter cycles controls the number of complete rainbow cycles, LED automatically turns off after completion.

```

static void prv_running_horse(uint32_t cycles)
{
    static const struct { uint8_t r, g, b; } colors[] = {
        {150, 0, 0}, // Red
        {150, 60, 0}, // Orange
        {150, 150, 0}, // Yellow
        {0, 150, 0}, // Green
        {0, 150, 150}, // Cyan
        {0, 0, 150}, // Blue
        {150, 0, 150}, // Purple
    };
    static const int color_count = sizeof(colors) / sizeof(colors[0]);

    for (uint32_t n = 0; n < cycles; n++)
    {
        for (int i = 0; i < color_count; i++)
        {
            led_strip_set_pixel(led_strip, 0,
                                colors[i].r, colors[i].g, colors[i].b);
            led_strip_refresh(led_strip);
            vTaskDelay(pdMS_TO_TICKS(200));
        }
        led_strip_clear(led_strip);
        led_strip_refresh(led_strip);
    }
}

```

## 5.6 app\_main() — Main Entry

Initializes GPIO button (pull-up input), WS2812 LED (via RMT peripheral driver), creates uint32\_t queue of length 30, then creates send task (bound to CPU1) and receive task (bound to CPU0). If queue creation fails, tasks are not started.

```

void app_main(void)
{
    gpio_config_t io_conf = {
        .pin_bit_mask = (1ULL << BUTTON_GPIO),
        .mode = GPIO_MODE_INPUT,

```



```

        .pull_up_en = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE,
    };
    gpio_config(&io_conf);

    led_strip_config_t strip_config = {
        .strip_gpio_num = LED_GPIO,
        .max_leds = LED_NUM,
        .led_model = LED_MODEL_WS2812,
        .color_component_format = LED_STRIP_COLOR_COMPONENT_FMT_GRB,
        .flags.invert_out = false,
    };
    led_strip_rmt_config_t rmt_config = {
        .resolution_hz = 10 * 1000 * 1000,
        .flags.with_dma = false,
    };
    ESP_ERROR_CHECK(led_strip_new_rmt_device(&strip_config, &rmt_config, &led_strip));
    led_strip_clear(led_strip);
    led_strip_refresh(led_strip);

    xQueue = xQueueCreate(QUEUE_LENGTH, sizeof(uint32_t));

    if (xQueue != NULL)
    {
        ESP_LOGI(TAG, "Queue created, press BOOT button to send.");

        xTaskCreatePinnedToCore(vSendTask, "Send", SEND_TASK_STACK_SIZE, NULL,
                                SEND_TASK_PRIORITY, NULL, 1);

        xTaskCreatePinnedToCore(vRecvTask, "Recv", RECV_TASK_STACK_SIZE, NULL,
                                RECV_TASK_PRIORITY, NULL, 0);
    }
}

```

## 6. Operation Flow

### 6.1 Compilation and Flashing Flow

For detailed flashing process: 1. Before Development → 3. Build and Flash

### 6.2 Normal Operation

1. After flashing, power on, wait for button input
2. **Press** BOOT button → Send success → Green LED lights for 300ms → Serial prints `Send OK`
3. Press 30 times consecutively → Send task prints `Send task STOP` and exits
4. Receive task takes one data from queue every 1 second
5. After receiving 30 → Receive task prints `Receive task STOP` → **Infinite rainbow marquee**

Log field meanings:

```
I (1263) queue_demo: Queue created, press BOOT button to send.
I (2000) queue_demo: Send OK: 0 | sent: 1/30          ← 1st press send
I (2300) queue_demo: Send OK: 1 | sent: 2/30          ← 2nd press send
I (3263) queue_demo: Receive: 0 | recv: 1/30         ← Received 1st data
I (4263) queue_demo: Receive: 1 | recv: 2/30         ← Received 2nd data
...
I (30000) queue_demo: Send task STOP. Total sent: 30
I (32000) queue_demo: Receive task STOP. Total received: 30
I (32000) queue_demo: Running horse light show start...
```

**0 pressure** — Rapidly press 30 times, sends to queue (buffered), receiver takes at its own pace (1/sec) = **Queue asynchronous decoupling.**

## 7. FAQ

| Symptom                             | Cause                                  | Solution                                                                |
|-------------------------------------|----------------------------------------|-------------------------------------------------------------------------|
| Send failed <code>Queue FULL</code> | Pressed too fast, queue already has 30 | Normal — queue capacity is limited, just wait for receiver to take data |
| Send success but green LED not lit  | LED refresh timing issue               | Confirm <code>led_strip_refresh</code> is called                        |
| Receive task not working            | Queue creation failed (NULL)           | Check <code>xqueueCreate</code> return value                            |
| Button insensitive                  | Debounce time inappropriate            | Adjust <code>DEBOUNCE_MS</code> (recommended 30ms)                      |
| Fast consecutive presses lose data  | Queue capacity insufficient            | Increase <code>QUEUE_LENGTH</code>                                      |
| Marquee colors abnormal             | GRB byte order error                   | Confirm <code>led_strip_set_pixel</code> parameter order is (R, G, B)   |